

ROBÓTICA

INFORME CAPÍTULO 6 parte 1 - RVC

Localización y mapeo

Docente: Jaime Enrique Arango Castro

Estudiante:

Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia

Ingeniería Electrónica

Sede Manizales



Manizales, Colombia

Junio 2025

1. Resumen

Este informe presenta los conceptos clave, algoritmos y simulaciones relacionadas con el Capítulo 6 del libro *Robotics, Vision and Control (RVC)* de Peter Corke. Se abordan técnicas de localización y mapeo, incluyendo odometría y , mapas de landmarks, filtros para estimación de posición y trayectorias. A su vez, se documenta el uso de las funciones realizadas en el cuadernillo de jupyter chap6.ipynb y su interpretación desde el punto de vista de la robótica móvil y su respectiva relación teórica.

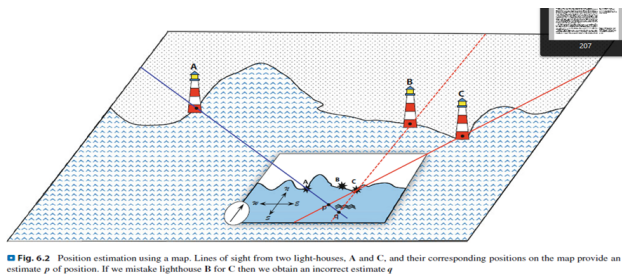


Figura 1: Utilidad de los mapas de puntos de referencia

La figura ilustra cómo un robot puede estimar su posición utilizando landmarks (referencias fijas conocidas) como faros. Midiendo los ángulos de rumbo (bearings) hacia estos landmarks, el robot puede triangular su posición.

Este método es esencial hoy en día para robots que operan sin GPS, enfrentando desafíos similares a los de los navegantes antiguos. Aun así, la identificación correcta de landmarks y medidas precisas son críticas para evitar errores en la localización. Es por ello que se puede estimar de las siguientes formas:

1.1. Reseccionamiento

Estimar la posición propia midiendo los ángulos hacia landmarks conocidos.

1.2. Triangulación

Estimar la ubicación de un punto desconocido midiendo los ángulos desde landmarks conocidos.

Nos damos cuenta de que, la información adicional de características observadas en el entorno es crítica para minimizar el error de la estimación de la pose de un robot, sin embargo, esta posibilidad del error en la información siempre estará presente. De aquí la importancia de simularlo.

1.3. FDPs

La localización del robot puede representarse como una función de densidad de probabilidad (PDF) sobre el espacio de posiciones posibles. Esta función indica qué tan probable es que el robot esté en cada punto (x,y) de nuestro sistema de coordenadas.

Estimar la PDF implica un proceso de estimación: inferir el estado verdadero (posición del robot) a partir de datos observados y conocimiento del modelo de movimiento.

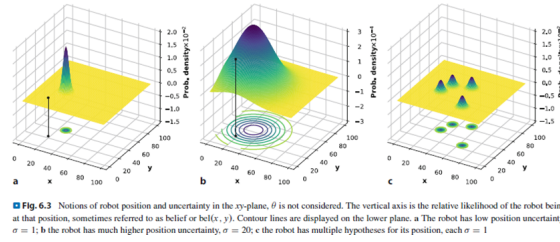


Figura 2: Probabilidad relativa como incertidumbre de la posición en el plano x-y

La incertidumbre se refleja en la forma de la FDP:

- PDF estrecha (σ bajo): alta certeza sobre la posición.
- PDF ancha (σ alto): alta incertidumbre.
- PDF multimodal: múltiples hipótesis sobre la ubicación.

Tal y como vemos, las PDFs permiten modelar la incertidumbre de manera explícita y son esenciales en técnicas como filtros de Bayes, Kalman o Monte Carlo para mejorar la precisión de la localización.

2. 6.1 Dead Reckoning usando Odometría

2.1. Descripción

Estimación incremental de la pose del robot basada en medidas internas como su pose, velocidad y tiempo adquiridas mediante mapeo de conexiones o sensores. (ruedas, IMU). Aunque simple, el error crece con el tiempo, esto debido al ruido del sensor y el deslizamiento de las ruedas.

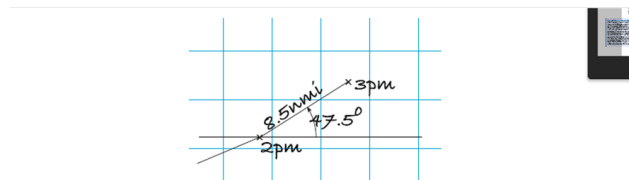


Figura 3: Estimación de posición usando dead reckoning.

Es importante recordar que debido a esta odometría imperfecta y suponiendo el error aditivo ya mencionado, tendremos un error o ruido de odometría con media cero y covarianza.

2.2. Implementación en python

2.2.1. Modelando el robot

De igual forma, veamos que esta covarianza se representa con una matriz cuyas diagonales son las varianzas o desviaciones estandar de la pose, es decir , tanto orientación como posición.

Cuando las variables son independientes, la varianza es cero.

```
V = np.diag([0.02, np.deg2rad(0.5)]) ** 2; # simulamos el ruido de sensores de odometria con una matriz de convarianza
V
[4] ✓ 0.4s
... array([[ 0.0004,      0],
        [      0, 7.615e-05]])

robot = Bicycle(covar=V, animation="car")
[10] ✓ 0.5s
... Bicycle: x = [ 0, 0, 0 ]
        L=1, steer_max=1.41372, speed_max=inf, accel_max=inf

odo = robot.step((1, 0.3)) # simulamos un paso con v lineal 1 y angulo de dirección 0.3 rad, esto entrega una odometria estimada al hacer el mov.
[11] ✓ 0.5s
... array([ 0.1025,  0.02978])

> robot.q # representa posicion y orientación, coordenadas (x,y) y orientación en radianes
[12] ✓ 0.0s
... array([ 0.1,      0,  0.03093])

robot.f([0, 0, 0], odo) # Aplica la función de traslación para predecir la nueva pose del robot desde [0,0,0] usando la odometria ya estimada.
[13] ✓ 0.0s
... array([ 0.1025,      0,  0.02978])

robot.control = RandomPath(workspace=10) # Se configura un controlador de movimiento aleatorio dentro de un espacio de trabajo de 10 unidades.
[14] ✓ 1.3s

> robot.run_animation(T=30) # Ejecuta la animación del robot durante 30 segundos, mostrando su movimiento basado en el controlador aleatorio configurado.
[15] ✓ 13.0s
... <matplotlib.animation.FuncAnimation at 0x19eca66e990>
```

Figura 4: Modelando el robot en PY.

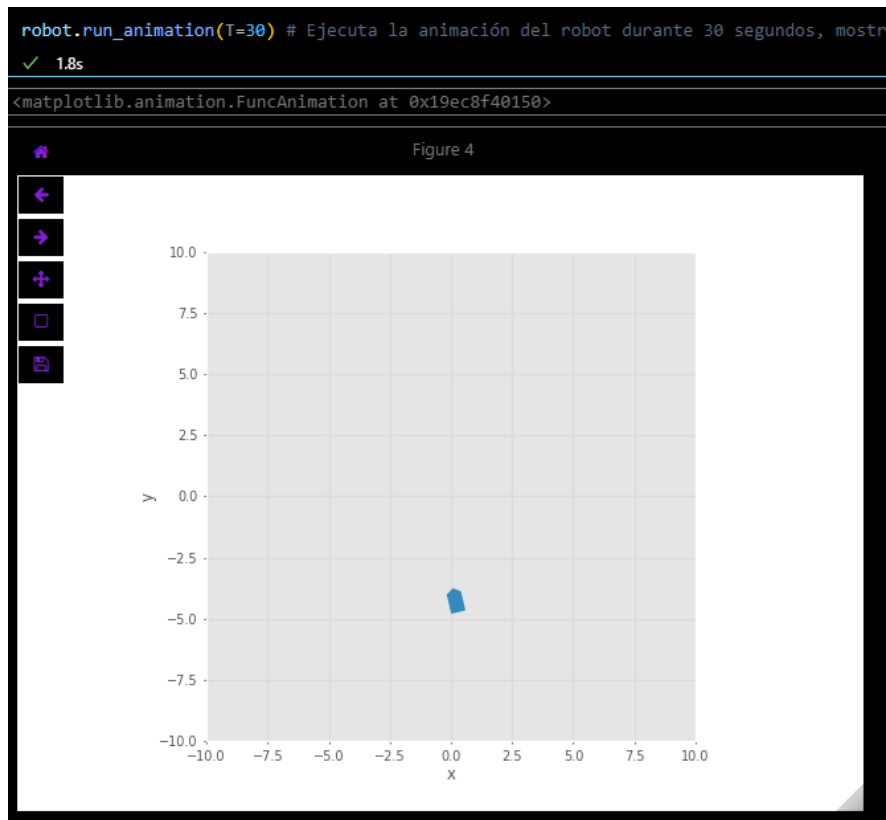


Figura 5: Posición final.

Vemos entonces que es algo complicado poder estimar con precisión la pose del robot en el espacio.

2.2.2. Estimar la pose

Estimación de la pose del robot usando medidas angulares o de distancia hacia puntos fijos conocidos (landmarks).

Principalmente, veamos que si tenemos variables independientes con varianza cero y covarianza cero, esto implica que si tenemos matrices jacobianas evaluadas con media cero, el valor del ruido no será medible.

```

> robot.Fx([0, 0, 0], [0.5, 0.1]) # se calcula el jacobiano de la función de movimiento del robot,
# es decir, la derivada de la función de movimiento respecto a la posición y orientación actual del
[19] ✓ 0.2s
... array([[ 1.,  0.,  0.],
          [ 0.,  1., 0.5],
          [ 0.,  0.,  1.]])

> x_sdev = [0.05, 0.05, np.deg2rad(0.5)];# covarianza inicial para el estado del robot
x_sdev
[21] ✓ 0.0s
... [0.05, 0.05, 0.008726646259971648]
```

Figura 6: Jacobiano y covarianza inicial.

De esta forma, observamos que la incertidumbre crece sin límites implementando estrategias como dead reckoning, es por esto que se usa información adicional del entorno para solucionar el problema al momento de localizar y mapear simultáneamente.

Una buena herramienta para estimación de poses no lineal es la versión extendida del filtro de kalman:

Uso de EKF

Fusiona odometría y observaciones para mejorar estimación,, puesto que estima la pose e incertidumbre por cada paso, reduciendo la incertidumbre representada por elipses de confiabilidad.

Primero definimos la matriz diagonal inicial del filtro y posteriormente creamos la versión extendida haciendo uso de nuestra clase EKF (extended Kalman Filter)

```
P0 = np.diag(x_sdev) ** 2; # matriz diagonal inicial del filtro de Kalman
P0
✓ 0.0s

array([[ 0.0025,    0,    0],
       [    0, 0.0025,    0],
       [    0,    0, 7.615e-05]])

# now create a extender version of kalman filter
ekf = EKF(robot=(robot, V), P0=P0)
✓ 0.3s

EKF object: 3 states, estimating: vehicle pose, map
robot: Bicycle: x = [ 0, 0, 0 ]
L=1, steer_max=1.41372, speed_max=inf, accel_max=inf
V_est: [ 0.0004, 0 | 0, 7.62e-05 ]
```

Figura 7: VERSION EXTENDIDA DEL FILTRO DE KALMAN.

Finalmente corremos la animación, y podemos observar también la trayectoria real en color azul, y la trayectoria estimada en rojo. De igual forma, podemos graficar las elipses en el trayecto las cuales representan la incertidumbre en la estimación, siendo mayor cada vez que pasa el tiempo.

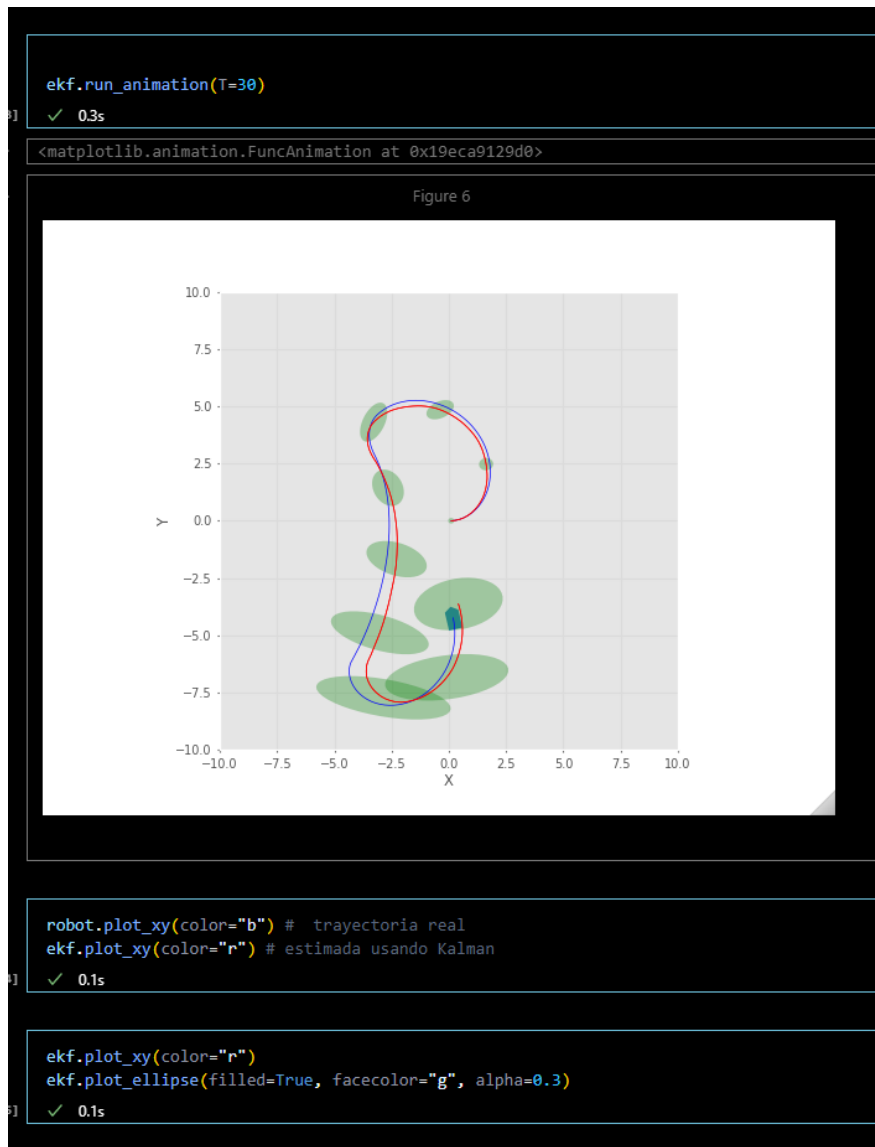


Figura 8: VERSION EXTENDIDA DEL FILTRO DE KALMAN.

Posteriormente nos interesa conocer la incertidumbre de la estimación, ergo ,obtenemos la varianza en el paso 150 y calculamos la desviación estandar de esta posición estimada. Tanto en x y y.

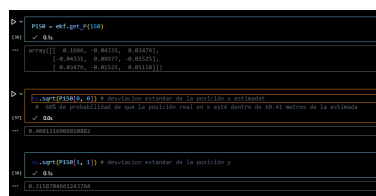


Figura 9: COVARIANZA Y DESVIACIÓN ESTANDAR

De donde, refiriendonos a la teoría conceptual de la distribución gaussiana el 68 porciento de los datos se encuentran dentro de ± 1 de desviación estandar al rededor de la media, es decir que , si el error en la estimación de nuestras coordenadas siguen la distribución normal supuesta por el error ya definido, hay un 68

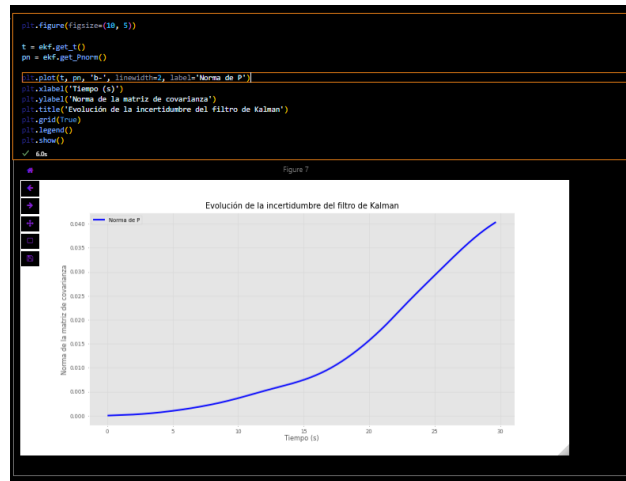


Figura 10: Covarianza y convergencia a través del tiempo

Finalmente, se obtienen los tiempos y la norma de la matriz de covarianza a lo largo del tiempo. Para posteriormente graficar la evolución de la incertidumbre y evaluar si el filtro converge o tiene problemas. De donde vemos claramente que el filtro no converge al pasar el tiempo, puesto que la matriz de covarianza calculada no logra estabilizarse en algun valor cte.

Es por esto que, tal como hemos dicho, la incertidumbre crece sin limites , esta es la razón de usar información adicional del entorno a la hora de hacer SLAM, donde los sensores MIDEN la probabilidad de averiguar donde se encuentra algo en cierto PUNTO de referencia ya definido.

3. 6.2 Localización con mapa de referencias (landmarks)

3.1. Descripción

- Se estiman las posiciones de los landmarks mientras se navega.
- Se extiende el vector de estado.
- Asumamos que el robot mide su posición con respecto a una referencia externa. Esta probabilidad será medida con un sensor, el cual tendrá una PDF Gaussiana relacionada mediante la multiplicación de la función de probabilidad usando odometría.

Combinar estimación por filtros de Kalman y la incertidumbre generada por un sensor, entrega una mejor estimación con menor incertidumbre a la pose del robot en el espacio.

3.2. Implementación en python

3.2.1. Creación mapa con puntos de referencia

Primero, creamos el mapa haciendo uso de la clase `LandmarkMap`, de donde creamos 20 puntos de referencia en un espacio de trabajo o sistema de coordenadas de 10x10.

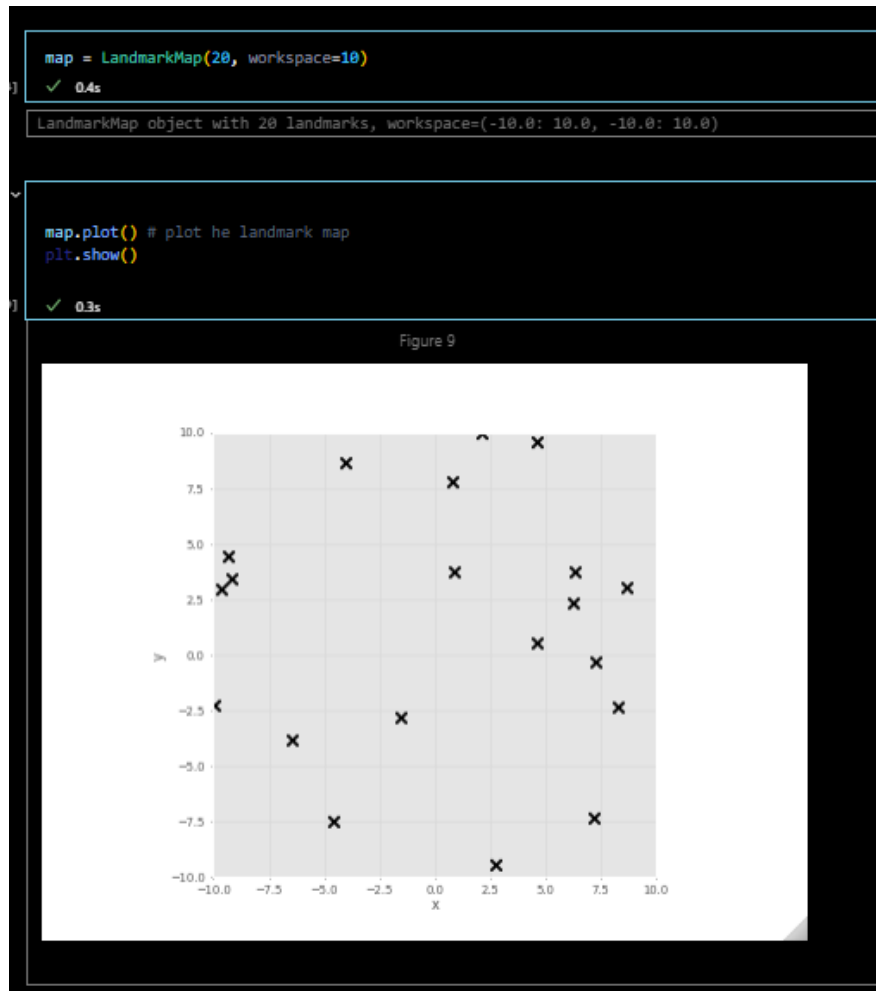


Figura 11: Modelando el mapa de puntos de referencia.

En robótica, los landmarks son características distintivas del entorno que el robot puede detectar y usar para localizarse. Este mapa simula un entorno con puntos de referencia que el robot podrá detectar con sus sensores. Es para este caso de trabajo, en el cual ahora se usan puntos de referencia conocidos para pulir la estimación de la pose del robot, esto se logra mediante reseccionamiento, o triangulación.

Continuamos entonces con la configuración del robot y el filtro de kalman extendido.

Definimos primero la matriz de covarianza que define el ruido de nuestros sensores en odometría, para posteriormente agregarlo a la definicion del sensor de rango, además de cubrir angulos desde -90 a 90 grados celsius y maximo de 4 unidades para el rango. De igual forma cambiamos la semilla del control para que la ruta sea aleatoria. De esta forma, observamos la posicion final en la que se encuentra.

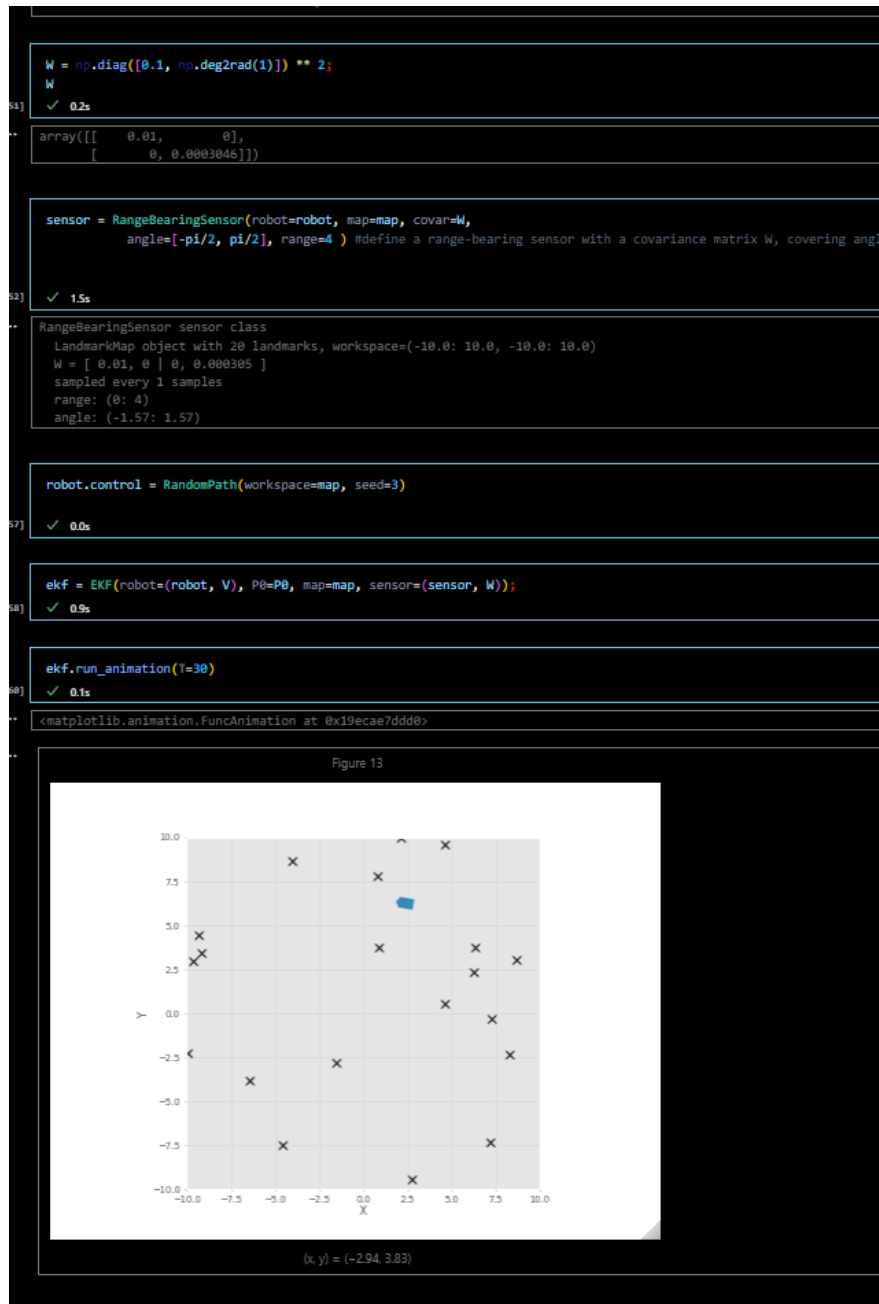


Figura 12: Modelando el sensor de rango.

Procedemos ahora entonces a realizar una lectura del sensor, tanto en rango como orientación a un punto de referencia. De donde, tomando la posición de nuestro punto de referencia, podemos inferir adecuadamente sobre esta estimación.

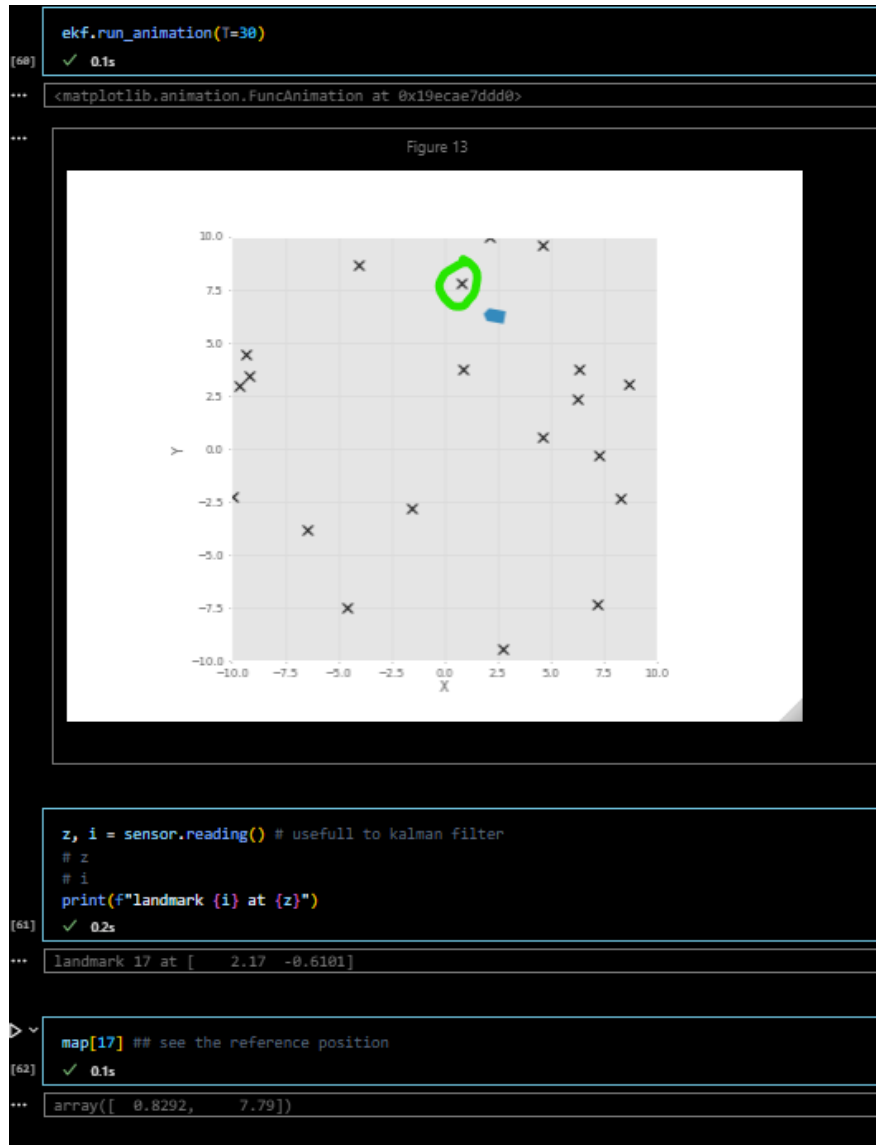


Figura 13: Modelando el sensor de rango.

3.2.2. Estimar la trayectoria

Finalmente, dibujamos la trayectoria de nuestro robot, y de igual forma las elipses de confiabilidad. De donde logramos observar que el combinar estimación por filtros de Kalman y la incertidumbre generada por el sensor, obtenemos una estimación correctiva con menor incertidumbre a la pose del robot, combinando adecuadamente esto facilmente visible en el tamaño de nuestras elipses.

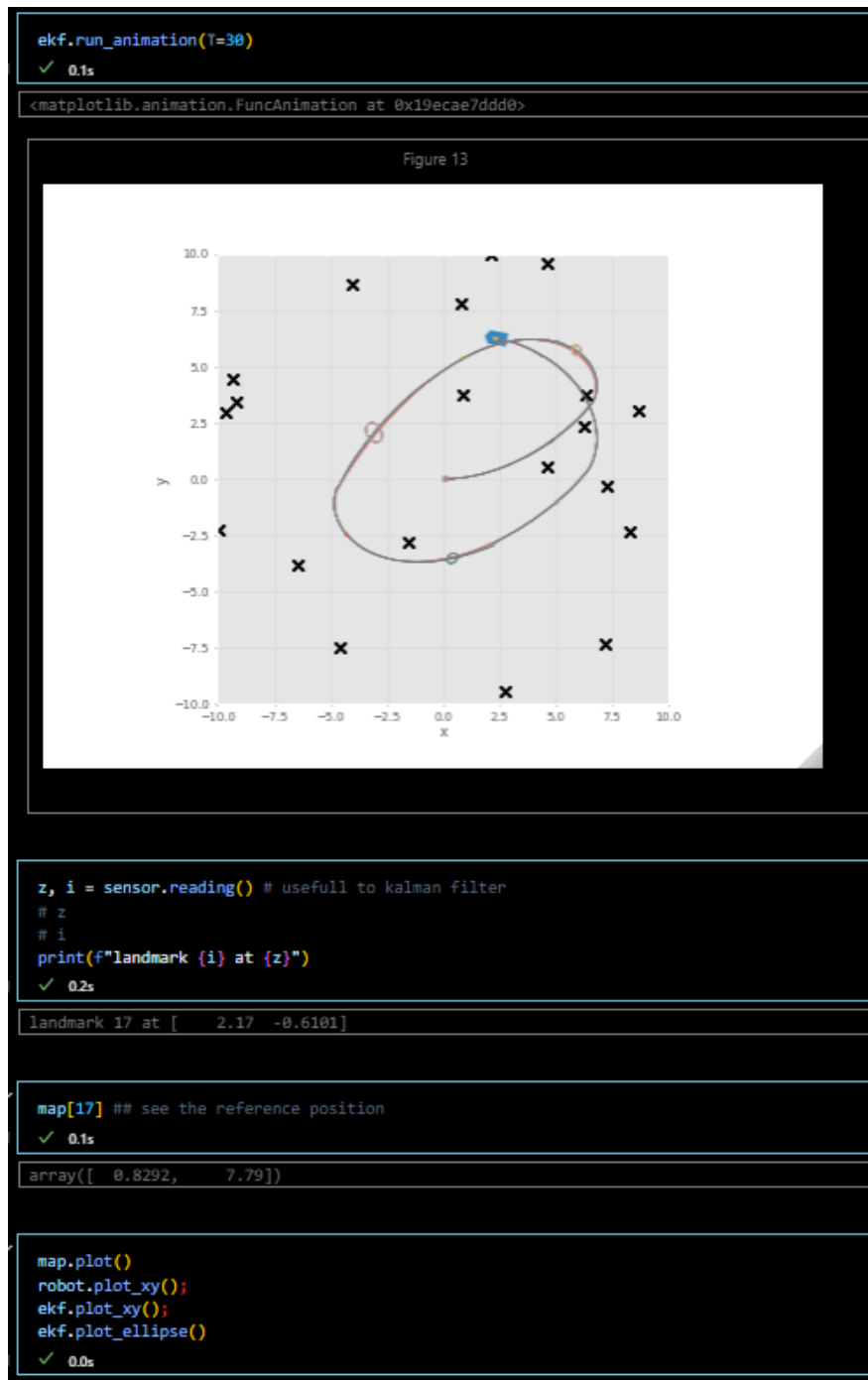


Figura 14: Estimación recursiva mediante Kalman y odometría.

Este proceso de corrección es recursivo, lo que significa que cada nueva medición ayuda a refinar la estimación anterior, permitiendo al filtro de Kalman actualizar la pose del robot con respecto a los puntos de referencia, resultando en elipses de confiabilidad más pequeñas y precisas cuando hay buenos puntos de referencia disponibles.

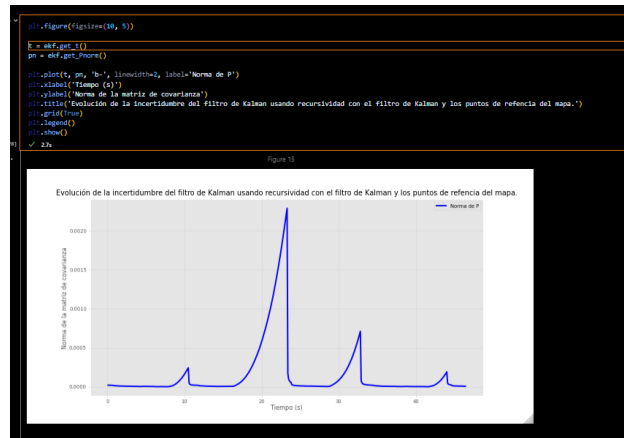


Figura 15: Covarianza y convergencia a través del tiempo

Finalmente, se obtienen los tiempos y la norma de la matriz de covarianza a lo largo del tiempo para ESTE SEGUNDO CASO.

4. 6.3 Creación de mapas de landmarks

En este punto, no es necesario estimar la posición del robot pero si los puntos de referencia. Asumimos de esta forma que, estos puntos se pueden identificar y además, que el robot cuenta con una localización perfecta.

A continuación se crea un mapa de puntos de referencia haciendo uso de las funciones definidas anteriormente compilando con diferentes características.

4.1. Configuración landmark

Se crea un mapa con 50 landmarks distribuidos aleatoriamente en un espacio de trabajo de 15x15 unidades. Cambiamos parámetro seei a 10 con el fin de observar cambios en las simulaciones.

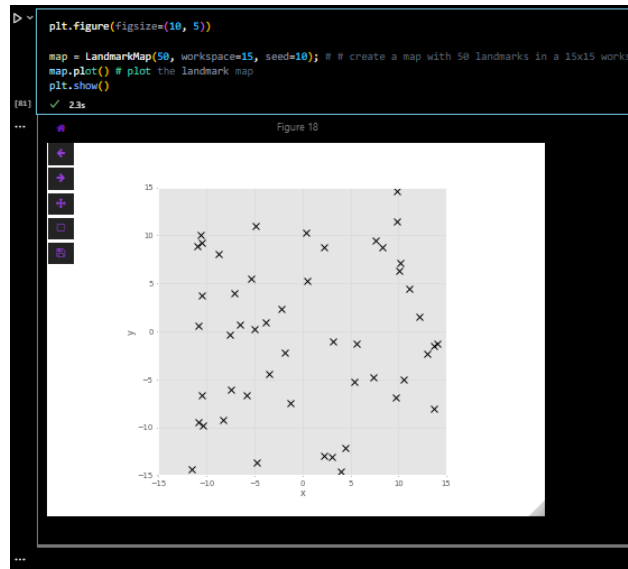


Figura 16: Mapa con 50 puntos de referencia en un sistema de coordenadas 15x15

4.2. Configuración del robot

Creamos un robot tipo vehiculo cuya covarianza sigue siendo la definida por el ruido de los sensores de odometría, para posteriormente crear una trayectoria aleatoria dentro del espacio de trabajo del mapa. Esto nos modela un robot con incertidumbre en su movimiento.

```

robot = Bicycle(covar=V, animation="car"); # create a robot with a covariance matrix V and a car-like animation

```

[82] ✓ 1.0s

```

robot.control = RandomPath(workspace=map, seed = 7); # set the robot's control to a random path within the map workspace with a seed for reproducibility

```

[83] ✓ 0.1s

Figura 17: Configuración robot

4.3. Sensor de rango y bearings

Definimos la matriz de covarianza para el sensor, con varianza de 0.1 y 1 grado en radianes para el ángulo. Esta representa la incertidumbre en las mediciones del sensor, lo cual es crucial para el filtro de Kalman. De igual manera, se procede con la definicion del sensor de rango, imputando todas las características necesarias ya definidas.

```

> W = np.diag([0.1, np.deg2rad(1)]) ** 2 # define the covariance matrix for the sensor, which includes both range and bearing noise
W
[86] ✓ 0.0s

array([[ 0.01,  0.],
       [ 0., 0.0003046]])

sensor = RangeBearingSensor(robot=robot, map=map, covar=W,
                             range=4, angle=[-pi/2, pi/2]); # create a range-bearing sensor with the specified covariance matrix, range, and angle limits
[88] ✓ 0.0s

> print(sensor)
[89] ✓ 0.0s

RangeBearingSensor sensor class
LandmarkMap object with 50 landmarks, workspace=(-15.0: 15.0, -15.0: 15.0)
W = [ 0.01, 0 | 0, 0.000305 ]
sampled every 1 samples
range: (0: 4)
angle: (-1.57: 1.57)

```

Figura 18: Sensor de rango y bearings

4.4. Filtro de Kalman extendido

Ahora continuamos con la definicion del filtro extendido de kalman usando nuestras definiciones de sensor y robot para cada caso. Es importante notar que ahora el robot no presenta una matriz de covarianza inicial, cuyas implicaciones dinámicas son importantes para tener en consideración.

```

ekf = EKF(robot=(robot, None), sensor=(sensor, W)); # create an Extended Kalman Filter (EKF) with the robot and sensor, without an initial covariance matrix for the robot
[90] ✓ 2.7s

> print(ekf)
[91] ✓ 0.1s

EKF object: 0 states, estimating: vehicle pose, map
robot: Bicycle: x = [ 0, 0, 0 ]
L=1, steer_max=1.41372, speed_max=inf, accel_max=inf
sensor: RangeBearingSensor sensor class
LandmarkMap object with 50 landmarks, workspace=(-15.0: 15.0, -15.0: 15.0)
W = [ 0.01, 0 | 0, 0.000305 ]
sampled every 1 samples
range: (0: 4)
angle: (-1.57: 1.57)
W est: [ 0.01, 0 | 0, 0.000305 ]

```

Figura 19: Filtro de Kalman extendido

4.5. correr la simulación y visualizar cambios por pantalla

Finalmente corremos la simulación y de esta manera, observamos los posibles cambios en pantalla, tales como

el final de trayectoria a la hora de realizar las 50 iteraciones

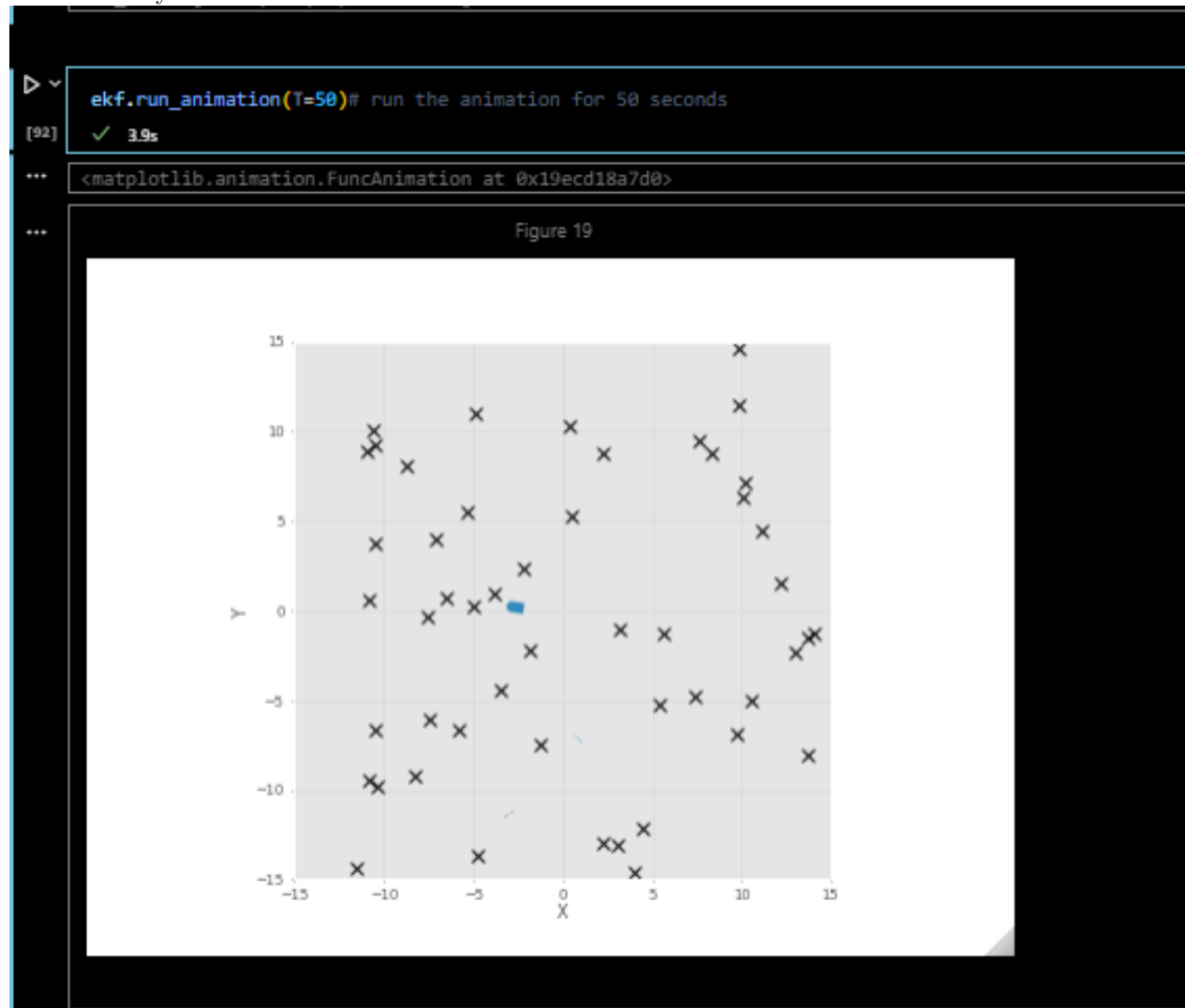


Figura 20: Final de la trayectoria

Se grafica el mapa real, el mapa estimado por el EKF y la trayectoria real del robot, permitiendo comparar visualmente la estimación con la realidad, evaluando el desempeño del algoritmo.

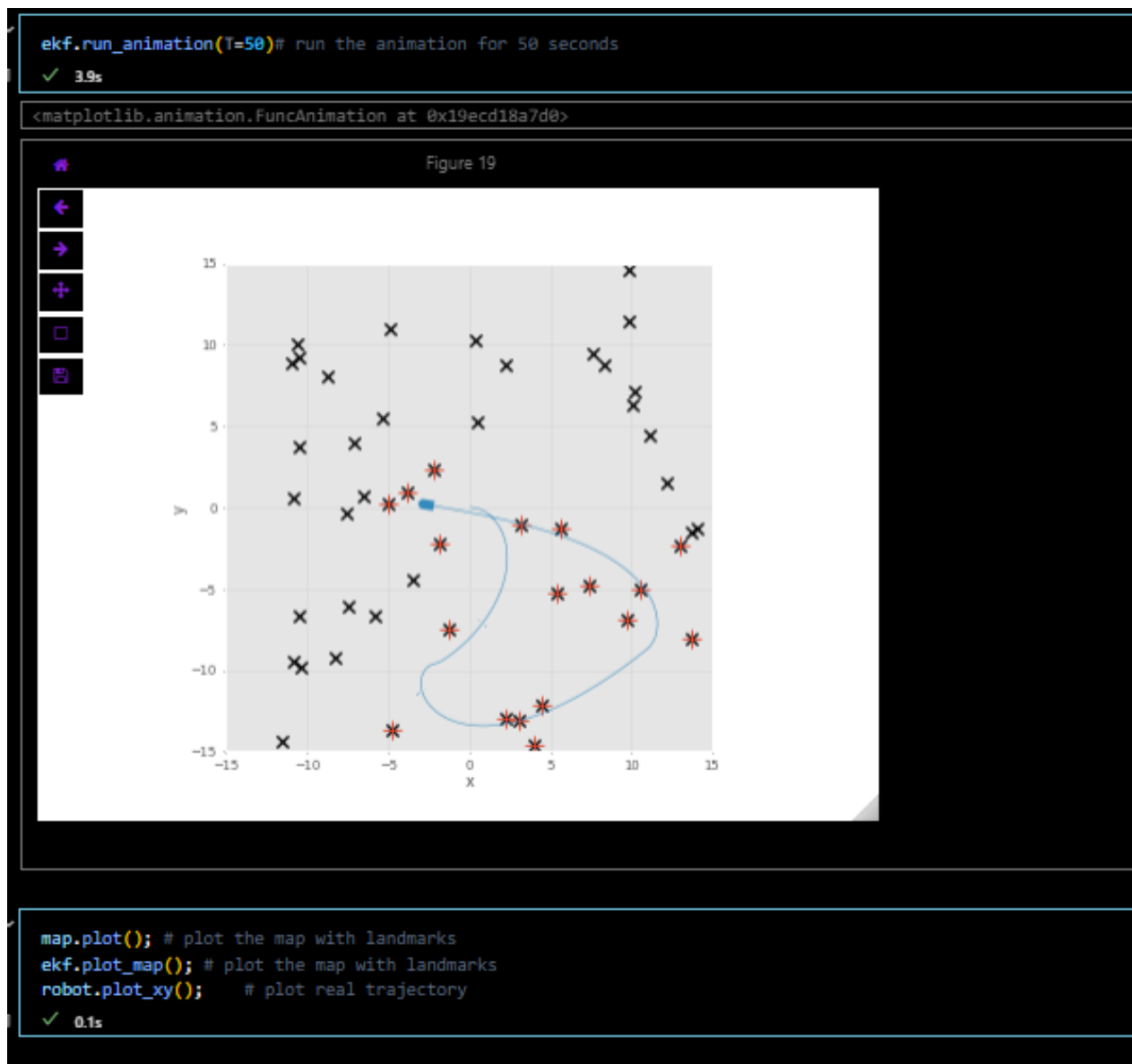
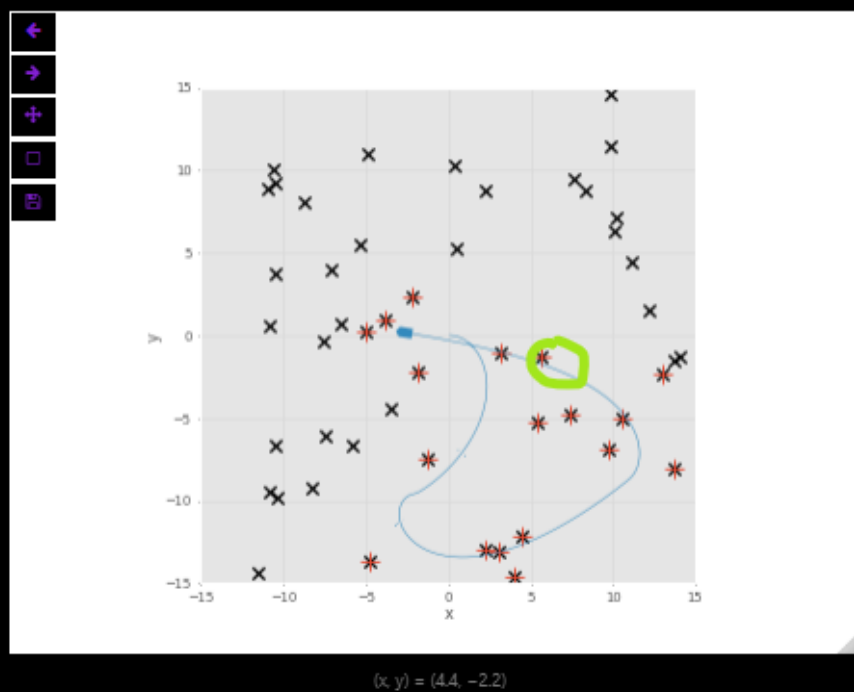


Figura 21: Simulación



```
map.plot(); # plot the map with landmarks
ekf.plot_map(); # plot the map with landmarks
robot.plot_xy(); # plot real trajectory
[93] ✓ 0.1s
```

```
i
[97] ✓ 0.4s
... 6
```

```
map[i]
[100] ✓ 0.3s
... array([ 5.671, -1.295])
```

```
ekf.landmark(i) # see how the landmark 6 is estimated
[96] ✓ 0.0s
... (1, 27)
```

```
ekf.x_est[24:26] # see the position of the landmark in the state vector, it is at the end of the state vector
[98] ✓ 0.0s
... array([ 13.03, -2.283])
```

```
ekf.P_est[24:26, 24:26] # print estimate position at this landmarks
[99] ✓ 0.0s
... array([[0.0008827, 0.000396],
          [0.000396, 0.001075]])
```

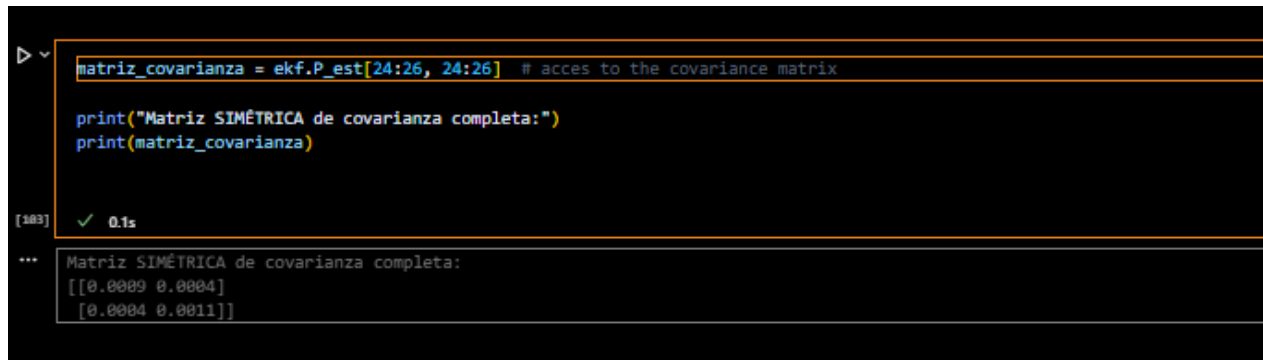
Figura 22: Simulación

Vemos la estimación de un landmark específico (el 6 en este caso)

El vector de estado del EKF contiene tanto la pose del robot como las posiciones de los landmarks, y la matriz P representa la incertidumbre en estas estimaciones

4.6. Matriz de covarianza simétrica

Es importante mencionar que la matriz de covarianza es simétrica.



```
matriz_covarianza = ekf.P_est[24:26, 24:26] # acces to the covariance matrix

print("Matriz SIMÉTRICA de covarianza completa:")
print(matriz_covarianza)
```

[183] ✓ 0.1s

... Matriz SIMÉTRICA de covarianza completa:
[[0.0009 0.0004]
 [0.0004 0.0011]]

Figura 23: matriz de covarianza

Esta matriz ayuda a cuantificar la incertidumbre en las estimaciones y también permite calcular los elipsoides de incertidumbre.

5. Relación con ROS2 (Robot Operating System 2)

Para el caso de ROS2, se pueden implementar distintos aspectos fundamentales del capítulo 6 (localización y mapeo) en el contexto de un vehículo autónomo móvil. A continuación, se enumeran los principales elementos y su correspondencia con funcionalidades ROS2. La implementación del LIDAR se deja a consideración

- **Dead Reckoning (Odometría):** Se implementa mediante un nodo que publique datos de odometría en el tópico `/odom`, utilizando el tipo de mensaje `nav_msgs/Odometry`. Este nodo puede recibir información de encoders o IMU.
- **Landmarks y Fusión Sensorial:** Los landmarks se pueden simular o detectar usando sensores (por ejemplo, LIDAR o cámara). La fusión con la odometría se realiza mediante el nodo `ekf_localization_node` del paquete `robot_localization`, que permite combinar múltiples fuentes en un estimador basado en filtros de Kalman extendidos.
- **SLAM:** Para construir simultáneamente un mapa y estimar la pose del robot, se puede usar `slam_toolbox` o `rtabmap_ros`. Estos paquetes toman como entrada mensajes de `/scan` (LIDAR) y `/odom`, y publican la estimación de mapa en `/map`.

- **Uncertainty y PDFs:** Aunque ROS2 no representa directamente distribuciones de probabilidad, estos conceptos se modelan en nodos como EKF, donde se maneja la incertidumbre de sensores y se propaga a través del sistema de localización.
- **SCAN:** Los datos de `semsadp` se publican en el tópico `/scan` como `sensor_msgs/scan`. Estos son esenciales para mapeo y evitación de obstáculos. ROS2 permite visualizar esto en tiempo real mediante `rviz2`.

Herramientas clave para la implementación en ROS2:

- manejo de nodos, topics, acciones y servicios `ros2 topic echo`, `ros2 run`, `ros2 launch`
- paquetes externos `robot_localization` para EKF
- `slam_toolbox` para mapeo
- `tf2` para manejar transformaciones entre marcos de referencia
- `rviz2` para visualización

Estas funcionalidades permiten implementar un sistema autónomo con capacidad de navegación, estimación de pose, percepción del entorno y mapeo, todos elementos esenciales en un entorno sin GPS, como se plantea en el capítulo.

6. Conclusiones

- La localización precisa es esencial para la navegación autónoma.
- SLAM permite operar en entornos sin mapas previos.
- La combinación de sensores (IMU, LIDAR), odometría y técnicas probabilísticas es clave
- ROS2 proporciona herramientas modulares como `nav_msgs/Odometry`, `tf2`, `robot_localization`, y `slam_toolbox` que permiten implementar algoritmos de odometría, SLAM y fusión sensorial, acercando la teoría de RVC al despliegue real en robots móviles.
- La correcta asociación de datos con landmarks, la reducción de la deriva odométrica y la estimación robusta de pose pueden lograrse mediante la integración de nodos ROS2, sensores (LIDAR, IMU), y filtros como EKF, permitiendo que el sistema funcione incluso en entornos dinámicos y sin GPS.

Referencias

- Corke, P. *Robotics, Vision and Control v2 (RVC)*
- notebook: `chap6.ipynb`
- Documentación de Robotics Toolbox
- GIT HUB RVC v2